SRI RAMAKRISHNA ENGINEERING COLLEGE

[Educational Service: SNR Sons Charitable Trust]
[Autonomous Institution, Reaccredited by NAAC with 'A+' Grade]
[Approved by AICTE and Permanently Affiliated to Anna University, Chennai] [ISO 9001:2015 Certified and all eligible programmes Accredited by NBA]
VATTAMALAIPALAYAM, N.G.G.O. COLONY POST,
COIMBATORE – 641 022

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

20CS253 FOUNDATIONS OF MACHINE LEARNING LABORATORY

LAB RECORD

ACADEMIC YEAR: 2024-2025

MAY 2025

KABINDRA PUN MAGAR **71812301170**

SRI RAMAKRISHNA ENGINEERING COLLEGE

[Educational Service : SNR Sons Charitable Trust]
[Autonomous Institution, Reaccredited by NAAC with 'A+' Grade]
[Approved by AICTE and Permanently Affiliated to Anna University, Chennai]
[ISO 9001:2015 Certified and all eligible programmes Accredited by NBA]
VATTAMALAIPALAYAM, N.G.G.O. COLONY POST, COIMBATORE – 641 022

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CLASS: **II B.E. (CSE)** SEMESTER: **IV**

Certified that this is the bonafide record of work done by Mr./Ms. Kabindra pun Magar In the **20CS253** **FOUNDATIONS OF MACHINE LEARNING LABORATORY** of this institution for IV Semester during the Academic year 2024-2025.

G. Priyanka 29/4/25
Faculty In-Charge
Mrs.G.Priyanka, AP(Sr.Gr)/CSE

MS Catherine 29/4/25
Professor & Head

ROLL NUMBER

71812301170

Submitted for the IV semester B.E. Practical Examination on_____.

Internal Examiner **Subject Expert**

INDEX

S.NO	DATE	TITLE	PAGE.NO	MARKS	STAFF SIGNATURE
1.	10.01.2025	ENTATION OF FIND – S ALGORITHM	1-3	9	
2.	24.01.2025	LEMENTATION OF CANDIDATE ELIMINATION ALGORITHM	4-6	9	
3.	31.01.2025	LEMENTATION OF LOCALLY WEIGHTED REGRESSION	7-9	9	
4.	07.02.2025	TATION OF DECISION TREE – ID3 ALGORITHM	10-14	10	
5.	14.02.2025	IMPLEMENTATION OF K-NEAREST NEIGHBOUR ALGORITHM	15-17	10	
6.	21.02.2025	ATION OF NAÏVE BAYES CLASSIFIER	18-20	10	
7.	28.02.2025	MPLEMENTATION OF NAÏVE BAYES FOR DOCUMENT CLASSIFICATION	21-23	10	
8.	07.03.2025	TATION OF BAYESIAN NETWORK	24-27	10	
9.	14.03.2025	ENTATION OF ARTIFICIAL NEURAL NETWORK	28-30	10	
10.	21.03.2025	MENTATION OF SUPPORT VECTOR MACHINE	31-33	10	
11.	4.04.2025	PLEMENTATION OF PRINCIPAL COMPONENT ANALYSIS AND SINGULAR VALUE DECOMPOSITION.	34-37	10	
		CONTENT BEYOND SYLLABUS			
12.	25.04.2025	STUDY ON ANN, RNN, CNN	38-40	—	

EX.NO:1	FIND-S ALGORITHM
DATE:24/1/25	

AIM:
To implement the Find-S algorithm for learning the most specific hypothesis in Machine Learning.

ALGORITHM:

STEP 1: Start the program.

STEP 2: Initialize the hypothesis h with the first positive training example.

STEP 3: For each remaining training example:

- If it is positive, compare it with h.
- Replace any differing attribute with ?.
- If it is negative, ignore it.

STEP 4: Repeat until all examples are processed.

STEP 5: Print the final hypothesis h.

STEP 6: Stop the program

PROGRAM:

```
import csv
hypo = ['%', '%', '%', '%', '%', '%']
with open('trainingdata.csv', newline='') as csv_file:
    readcsv = csv.reader(csv_file, delimiter=',')
    next(readcsv)
    data = []
    print("\nThe given training examples are:")
    for row in readcsv:
        print(row)
        if row[len(row) - 1].upper() == "YES":
            data.append(row)
print("\nThe positive examples are:")
for example in data:
```

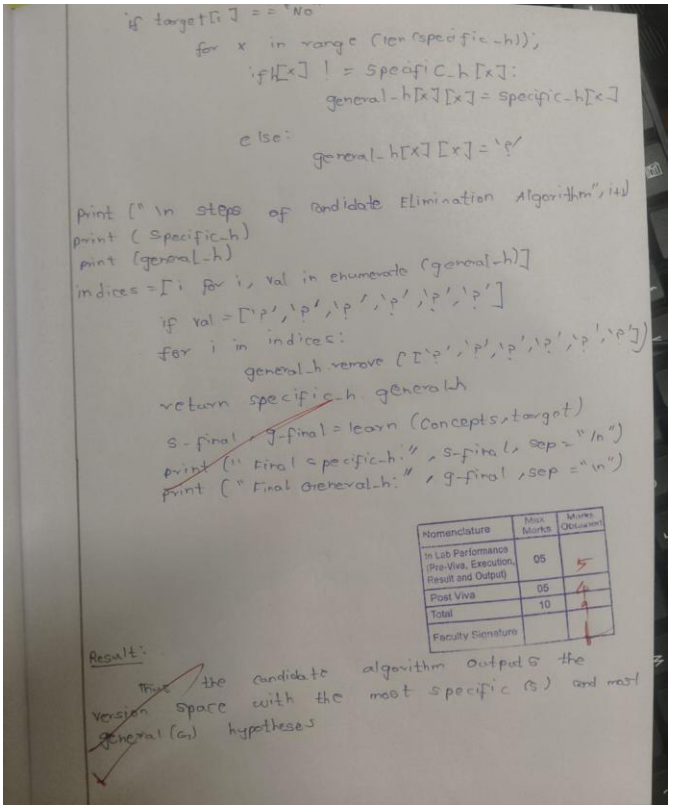
```
    print(example)
TotalExamples = len(data)
i = 0
j = 0
k = 0
print("\nThe steps of the Find-S algorithm are:\n", hypo)
list = []
d = len(data[0]) - 1
for j in range(d):
    list.append(data[i][j])
hypo = list
i = 1
for i in range(TotalExamples):
    for k in range(d):
        if hypo[k] != data[i][k]:
            hypo[k] = '?'
    print(hypo)
print("\nThe maximally specific Find-S hypothesis for the given training examples is:")
final_hypo = []
for i in range(d):
    final_hypo.append(hypo[i])
print(final_hypo)
```

DATASET:

```
Colour,Toughness,Fungus,Appearance,Poisonous
Orange,Hard,Yes,Wrinkled,No
Green,Hard,Yes,Wrinkled,Yes
Green,Hard,Yes,Wrinkled,Yes
Green,Soft,No,Smooth,Yes
Brown,Hard,No,Smooth,No
Orange,Hard,No,Wrinkled,No
Green,Hard,Yes,Wrinkled,Yes
```

OUTPUT:

```
The given training examples are:
['Orange', 'Hard', 'Yes', 'Wrinkled', 'No']
['Green', 'Hard', 'Yes', 'Wrinkled', 'Yes']
['Green', 'Hard', 'Yes', 'Wrinkled', 'Yes']
['Green', 'Soft', 'No', 'Smooth', 'Yes']
['Brown', 'Hard', 'No', 'Smooth', 'No']
['Orange', 'Hard', 'No', 'Wrinkled', 'No']
['Green', 'Hard', 'Yes', 'Wrinkled', 'Yes']

The positive examples are:
['Green', 'Hard', 'Yes', 'Wrinkled', 'Yes']
['Green', 'Hard', 'Yes', 'Wrinkled', 'Yes']
['Green', 'Soft', 'No', 'Smooth', 'Yes']
['Green', 'Hard', 'Yes', 'Wrinkled', 'Yes']

The steps of the Find-S algorithm are:
 ['%', '%', '%', '%', '%', '%']
['Green', 'Hard', 'Yes', 'Wrinkled']
['Green', 'Hard', 'Yes', 'Wrinkled']
['Green', '?', '?', '?']
['Green', '?', '?', '?']

The maximally specific Find-S hypothesis for the given training examples is:
['Green', '?', '?', '?']
```



```
if target[i] == 'No'
    for x in range (len(specific_h));
        if[x] != specific_h[x]:
            general_h[x][x] = specific_h[x]
        else:
            general_h[x][x] = '?'

print ("\n steps of Candidate Elimination Algorithm", i+1)
print ( specific_h)
print (general_h)
indices = [i for i, val in enumerate (general_h)]
    if val = ['?','?','?','?','?','?']
    for i in indices:
        general_h remove ([['?','?','?','?','?','?']])
return specific_h, general_h
s_final, g_final = learn (Concepts, target)
print ("Final specific_h:", s_final, sep = "\n")
print ("Final General_h:", g_final, sep = "\n")
```

Nomenclature	Max Marks	Marks Obtained
In Lab Performance (Pre-Viva, Execution, Result and Output)	05	5
Post Viva	05	4
Total	10	9
Faculty Signature		

Result:

True the candidate algorithm output's the version space with the most specific (S) and most general (G) hypotheses

RESULT:

The Find-S algorithm outputs the most specific hypothesis that fits all positive example.

EX.NO:2	CANDIDATE ELIMINATION ALGORITHM
DATE:31/1/25	

AIM:
To implement the Candidate Elimination algorithm to find the version space between the most specific and most general hypotheses.

ALGORITHM:

STEP 1: Start the program.

STEP 2: Initialize:

- S as the first positive training example.
- G as the most general hypothesis (all ?).

STEP 3: For each training example:

- If positive → Generalize S where needed.
- If negative → Specialize G to exclude the example.

STEP 4: Remove inconsistent hypotheses from G.

STEP 5: Repeat until all examples are processed.

STEP 6: Print the final version space (S and G).

STEP 7: Stop the program

PROGRAM:

```
import numpy as np

import pandas as pd

data = pd.DataFrame(data=pd.read_csv('candidate.csv'))

print(data)

concepts = np.array(data.iloc[:, 0:-1])

print(concepts)

target = np.array(data.iloc[:, -1])

print(target)

def learn(concepts, target):

    specific_h = concepts[0].copy()

    print("\nInitialization of specific_h and general_h")

    print(specific_h)
```

```
    general_h = [["?" for i in range(len(specific_h))] for i in range(len(specific_h))]
    print(general_h)
    for i, h in enumerate(concepts):
        if target[i] == "Yes":
            for x in range(len(specific_h)):
                if h[x] != specific_h[x]:
                    specific_h[x] = '?'
                    general_h[x][x] = '?'
        if target[i] == "No":
            for x in range(len(specific_h)):
                if h[x] != specific_h[x]:
                    general_h[x][x] = specific_h[x]
                else:
                    general_h[x][x] = '?'
        print("\nSteps of Candidate Elimination Algorithm", i+1)
        print(specific_h)
        print(general_h)
    indices = [i for i, val in enumerate(general_h) if val == ['?', '?', '?', '?', '?', '?']]
    for i in indices:
        general_h.remove(['?', '?', '?', '?', '?', '?'])
    return specific_h, general_h
s_final, g_final = learn(concepts, target)
print("\nFinal Specific_h:", s_final, sep="\n")
print("\nFinal General_h:", g_final, sep="\n")
```

DATASET:

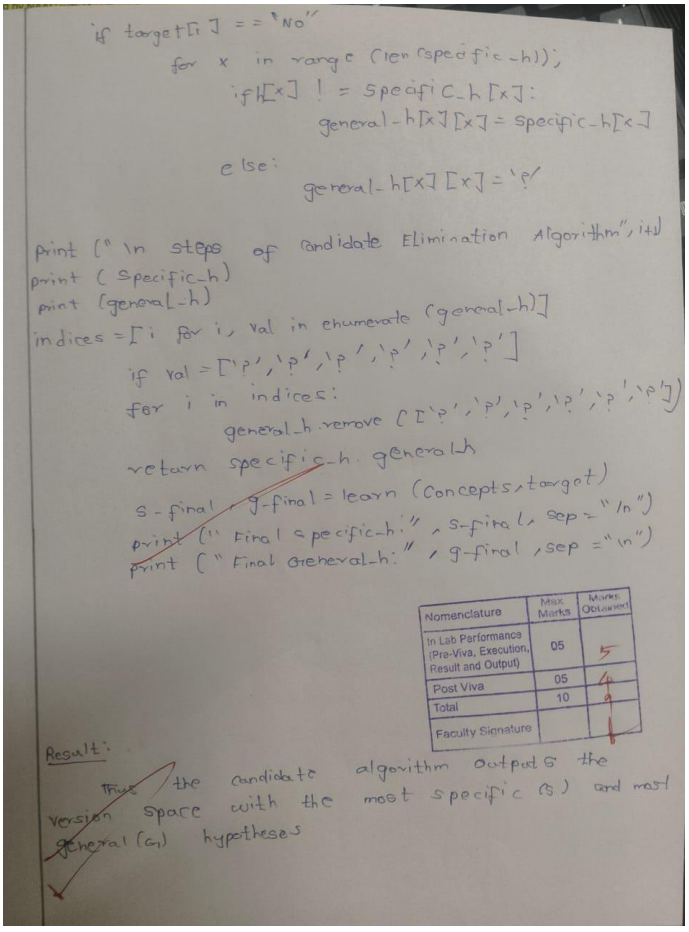
```
Age,Income,Occupation,Buys_Lap
Young,High,Student,Yes
Middle,Low,Professional,No
Old,Medium,Retired,Yes
Young,Low,Student,No
```

Middle,High,Professional,Yes

OUTPUT:

```
      Age  Income    Occupation Buys Laptop (Class)
0   Young    High       Student                 Yes
1  Middle     Low  Professional                  No
2     Old  Medium       Retired                 Yes
3   Young     Low       Student                  No
4  Middle    High  Professional                 Yes
[['Young' 'High' 'Student']
 ['Middle' 'Low' 'Professional']
 ['Old' 'Medium' 'Retired']
 ['Young' 'Low' 'Student']
 ['Middle' 'High' 'Professional']]
['Yes' 'No' 'Yes' 'No' 'Yes']

Initialization of specific_h and general_h
['Young' 'High' 'Student']
[['?', '?', '?'], ['?', '?', '?'], ['?', '?', '?']]

Steps of Candidate Elimination Algorithm 1
['Young' 'High' 'Student']
[['?', '?', '?'], ['?', '?', '?'], ['?', '?', '?']]

Steps of Candidate Elimination Algorithm 2
['Young' 'High' 'Student']
[['Young', '?', '?'], ['?', 'High', '?'], ['?', '?', 'Student']]

Steps of Candidate Elimination Algorithm 3
['?' '?' '?']
[['?', '?', '?'], ['?', '?', '?'], ['?', '?', '?']]
```

```
Steps of Candidate Elimination Algorithm 4
['?' '?' '?']
[['?', '?', '?'], ['?', '?', '?'], ['?', '?', '?']]

Steps of Candidate Elimination Algorithm 5
['?' '?' '?']
[['?', '?', '?'], ['?', '?', '?'], ['?', '?', '?']]

Final Specific_h:
['?' '?' '?']

Final General_h:
[['?', '?', '?'], ['?', '?', '?'], ['?', '?', '?']]
```



```
if target[i] == "No"
        for x in range (len(specific_h)):
                if h[x] != specific_h[x]:
                        general_h[x][x] = specific_h[x]
        else:
                        general_h[x][x] = '?'

print ("In steps of Candidate Elimination Algorithm", i+1)
print (specific_h)
print (general_h)
indices = [i for i, val in enumerate (general_h)]
        if val = ['?','?','?','?','?','?']
        for i in indices:
                general_h.remove ([['?','?','?','?','?','?']])
        return specific_h, general_h
        s_final, g_final = learn (Concepts, target)
        print ("Final specific_h:", s_final, sep = "\n")
        print ("Final General_h:", g_final, sep = "\n")
```

Nomenclature	Max Marks	Marks Obtained
In Lab Performance (Pre-Viva, Execution, Result and Output)	05	5
Post Viva	05	4
Total	10	9
Faculty Signature		

Result:

Thus the candidate algorithm outputs the version space with the most specific (S) and most general (G) hypotheses

RESULT:

The Candidate Elimination algorithm outputs the version space with the most specific (S) and most general (G) hypotheses.

EXP NO: 3	LOCALLY WEIGHTED REGRESSION ALGORITHM
DATE:07/02/2025	

AIM:

To implement Locally Weighted Regression algorithm using a dataset.

ALGORITHM :

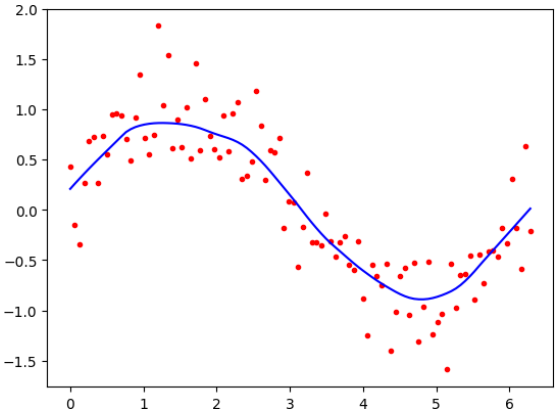
Step 1: Compute the smoothing neighborhood and calculate tricube weights for each datapoint.
Step 2: Initialize weight adjustments (delta), estimated values (yest), and set the iteration count.
Step 3: Perform weighted linear regression using least squares to fit a local model for each point.
Step 4: Update robustness weights (delta) based on residuals and median absolute deviation to downweight outliers.
Step 5: Iterate the regression and weight update steps until convergence, then return the final smoothed values

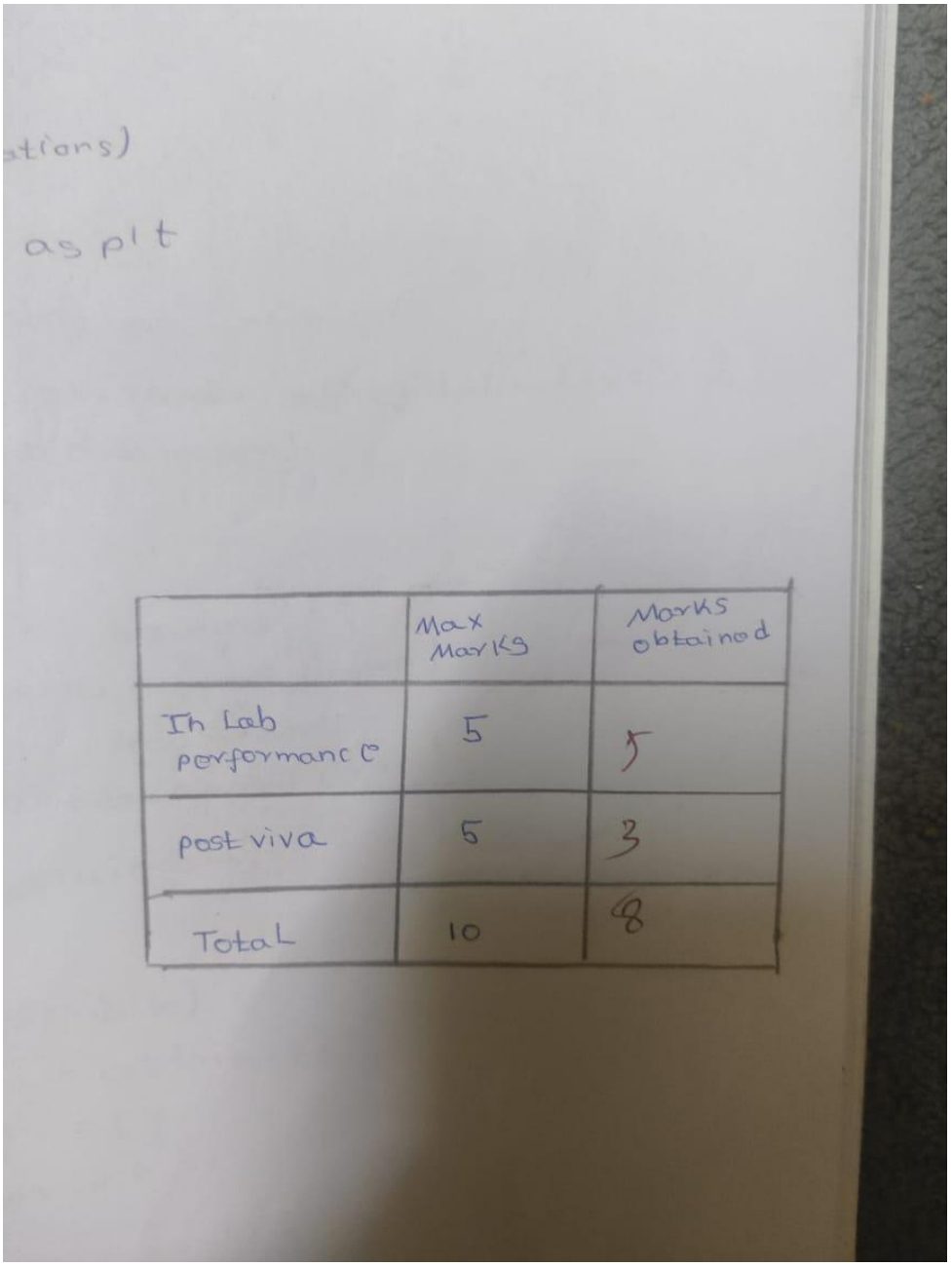
PROGRAM

```
from math import ceil
import numpy as np
from scipy import linalg
def lowess(x, y, f, iterations):
    n = len(x)
    r = int(ceil(f * n))
    h = [np.sort(np.abs(x - x[i]))[r] for i in range(n)]
    w = np.clip(np.abs((x[:, None] - x[None, :]) / h), 0.0, 1.0)
    w = (1 - w ** 3) ** 3
    yest = np.zeros(n)
    delta = np.ones(n)
    for iteration in range(iterations):
```

```
        for i in range(n):
            weights = delta * w[:, i]
            b = np.array([np.sum(weights * y), np.sum(weights * y * x)])
            A = np.array([[np.sum(weights), np.sum(weights * x)], [np.sum(weights * x),
np.sum(weights * x * x)]])
            beta = linalg.solve(A, b)
            yest[i] = beta[0] + beta[1] * x[i]
        residuals = y - yest
        s = np.median(np.abs(residuals))
        delta = np.clip(residuals / (6.0 * s), -1, 1)
        delta = (1 - delta ** 2) ** 2
    return yest
import math
n = 100
x = np.linspace(0, 2 * math.pi, n)
y = np.sin(x) + 0.3 * np.random.randn(n)
f = 0.25
iterations = 3
yest = lowess(x, y, f, iterations)
import matplotlib.pyplot as plt
plt.plot(x, y, "r.")
plt.plot(x, yest, "b-")
plt.show()
```

OUTPUT:





ations)

as plt

	Max Marks	Marks obtained
In Lab performance	5	5
post viva	5	3
Total	10	8

RESULT:

Thus, the program to implement locally weighted regression Algorithm is executed successfully.

EXP NO: 4	ID3 ALGORITHM
DATE:21/02/2025	

AIM:

To implement ID 3 algorithm using a dataset.

ALGORITHM :

Step 1: Read the CSV file and load the dataset into metadata (headers) and training data.
Step 2: Compute entropy for the dataset and calculate gain ratio for each attribute**.**
Step 3: Select the attribute with the highest gain ratio as the decision node.
Step 4: Split the dataset based on the selected attribute and create child nodes for each subset.
Step 5: Recursively repeat steps 2-4 for each child node until all samples in a subset belong to the same class.
Step 6: Construct the decision tree with attributes as nodes and classification labels as leaf nodes.
Step 7: Print the final decision tree in a structured format

PROGRAM

```
import numpy as np

import math

import csv

def read_data(filename):

    with open(filename, 'r') as csvfile:

        datareader = csv.reader(csvfile, delimiter=',')

        headers = next(datareader)

        metadata = []

        traindata = []

        for name in headers:

            metadata.append(name)
```

```
        for row in datareader:
            traindata.append(row)
    return (metadata, traindata)
class Node:
    def _init_(self, attribute):
        self.attribute = attribute
        self.children = []
        self.answer = ""
    def _str_(self):
        return self.attribute
def subtables(data, col, delete):
    dict = {}
    items = np.unique(data[:, col])
    count = np.zeros(items.shape, dtype=np.int32)
    for i, item in enumerate(items):
        count[i] = np.sum(data[:, col] == item)
    for i, item in enumerate(items):
        dict[item] = np.empty((count[i], data.shape[1]), dtype=object)
        pos = 0
        for row in data:
            if row[col] == item:
                dict[item][pos] = row
                pos += 1
    if delete:
        for item in items:
            dict[item] = np.delete(dict[item], col, 1)
    return items, dict
def entropy(S):
    items = np.unique(S)
    if len(items) == 1:
```

```
        return 0
    counts = np.array([np.sum(S == item) for item in items], dtype=np.float64) / len(S)
    return -np.sum(counts * np.log2(counts))
def gain_ratio(data, col):
    items, dict = subtables(data, col, delete=False)
    total_size = data.shape[0]
    entropies = np.array([entropy(dict[item][:, -1]) * dict[item].shape[0] / total_size for item in
items])
    intrinsic_values = -np.array([dict[item].shape[0] / total_size * np.log2(dict[item].shape[0] /
total_size) for item in items])
    iv = np.sum(intrinsic_values)
    total_entropy = entropy(data[:, -1])
    return (total_entropy - np.sum(entropies)) / iv
def create_node(data, metadata):
    if len(np.unique(data[:, -1])) == 1:
        node = Node("")
        node.answer = np.unique(data[:, -1])[0]
        return node
    gains = np.array([gain_ratio(data, col) for col in range(data.shape[1] - 1)])
    split = np.argmax(gains)
    node = Node(metadata[split])
    metadata = np.delete(metadata, split, 0)
    items, dict = subtables(data, split, delete=True)
    for item in items:
        child = create_node(dict[item], metadata)
        node.children.append((item, child))
    return node
def empty(size):
    return " " * size
def print_tree(node, level):
    if node.answer:
```

```
        print(empty(level), node.answer)
    else:
        print(empty(level), node.attribute)
         for value, child in node.children:
            print(empty(level + 1), value)
             print_tree(child, level + 2)
metadata = ["Age", "Gender", "Education", "Employment", "Location", "Marital Status", "Target"]
traindata = [
    ["Young", "Male", "Bachelor", "Full-time", "Urban", "Single", "Positive"],
    ["Middle", "Female", "Master", "Part-time", "Suburban", "Married", "Negative"],
    ["Old", "Male", "PhD", "Retired", "Rural", "Widowed", "Negative"],
    ["Young", "Female", "Bachelor", "Full-time", "Urban", "Single", "Positive"]
]
data = np.array(traindata, dtype=object)
node = create_node(data, metadata)
print_tree(node, 0)
```

OUTPUT:

```
Age
  Middle
        Negative
  Old
        Negative
  Young
        Positive
```

+1) , Value)

(+2)

("termisdata.csv")

a)

	Max Marks	Marks obtained
In Lab performance	5	5
post viva	5	3
Total	10	8

xecute ID3 algorithm has

ed

RESULT:

Thus, the program to implement ID 3 Algorithm is executed successfully.

EX.NO : 5	K- NEAREST NEIGHBOUR ALGORITHM
DATE : 28.02.25	

AIM :

To implement K – Nearest Neighbour algorithm using a dataset.

ALGORITHM :

Step 1: Load the Iris dataset and extract features (iris.data) and labels (iris.target).

Step 2: Split the dataset into training (80%) and testing (20%) sets using train_test_split().

Step 3: Initialize the K-Nearest Neighbors classifier with n_neighbors=6 and train it using fit().

Step 4: Predict the labels for the test set using predict().

Step 5: Evaluate the model's performance using classification_report() and print theaccuracy.

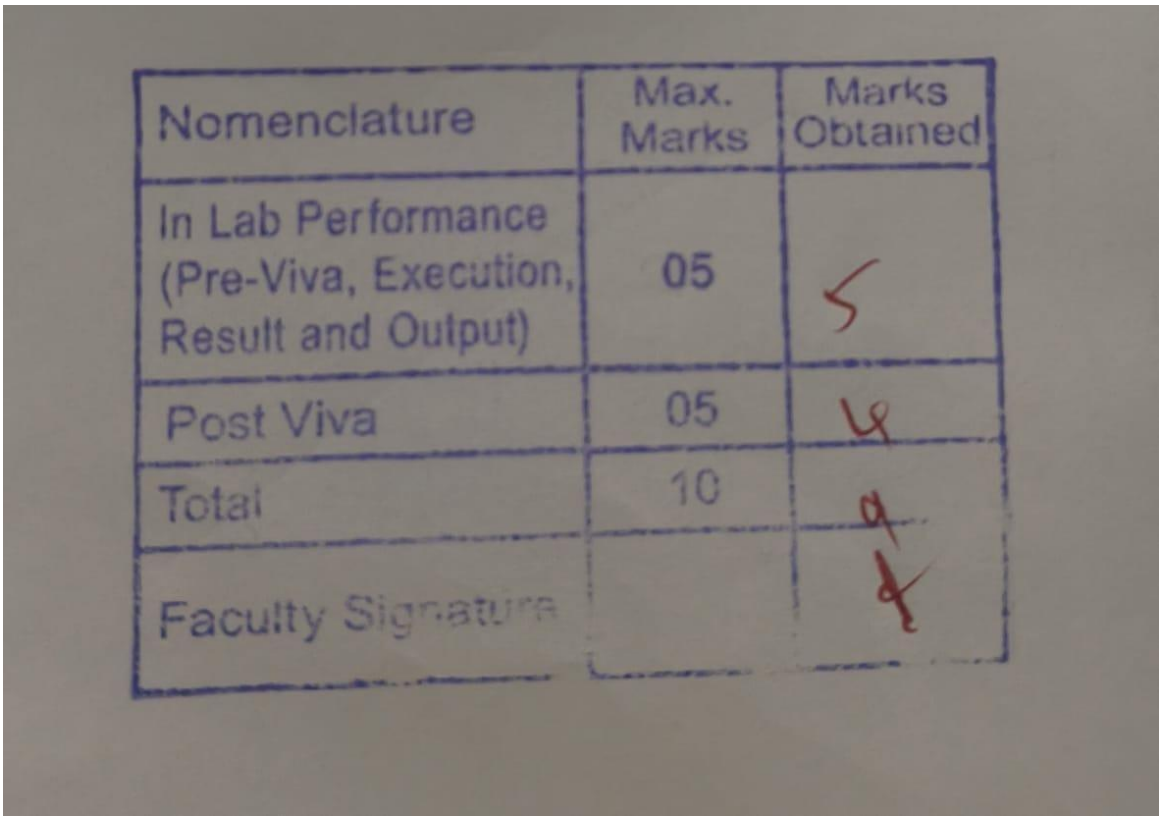
PROGRAM :

```
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix
import pandas as pd
import numpy as np
from sklearn import datasets
iris=datasets.load_iris()
iris_data=iris.data
iris_labels=iris.target
x_train, x_test, y_train, y_test=(train_test_split(iris_data, iris_labels, test_size=0.20))
classifier=KNeighborsClassifier(n_neighbors=6)
classifier.fit(x_train, y_train)
y_pred=classifier.predict(x_test)
print("accuracy is")
```

```
print(classification_report(y_test, y_pred))
```

OUTPUT :

```
accuracy is
              precision    recall  f1-score   support

           0       1.00      1.00      1.00         9
           1       1.00      1.00      1.00        12
           2       1.00      1.00      1.00         9

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```



Nomenclature	Max. Marks	Marks Obtained
In Lab Performance (Pre-Viva, Execution, Result and Output)	05	5
Post Viva	05	4
Total	10	9
Faculty Signature		

RESULT :

Thus, the program to implement K – Nearest Neighbour Algorithm is executed successfully.

EX.NO : 6	NAÏVE BAYESIAN CLASSOFIER ALGORITHM
DATE : 07.03.25	

AIM :

To implement naïve Bayesian classifier algorithm using a dataset.

ALGORITHM :

Step 1: Create a dataset as a Pandas DataFrame with categorical features and a target column.

Step 2: Split the dataset into feature variables (X) and target variable (y), excluding unnecessary columns.

Step 3: Print the first five rows of the dataset, feature variables, and target variable for verification.

Step 4: Encode categorical variables into numeric format using LabelEncoder.

Step 5: Print the transformed feature set and encode the target variable into numeric values.

Step 6: Split the dataset into training and testing sets using train_test_split().

Step 7: Initialize and train a Naïve Bayes classifier (GaussianNB) using the training set.

Step 8: Predict the test set labels and evaluate the model's accuracy using accuracy_score()

PROGRAM :

```
# Import necessary libraries
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
# Define the dataset as a DataFrame (normally, this would be read from a CSV)
data = pd.DataFrame({
    'Age': ['Young', 'Middle', 'Old', 'Young'],
    'Gender': ['Male', 'Female', 'Male', 'Female'],
    'Education': ['Bachelor', 'Master', 'PhD', 'Bachelor'],
    'Employment': ['Full-time', 'Part-time', 'Retired', 'Full-time'],
    'Location': ['Urban', 'Suburban', 'Rural', 'Urban'],
```

```
    'Marital Status': ['Single', 'Married', 'Widowed', 'Single'],
    'Target': ['Positive', 'Negative', 'Negative', 'Positive']  # The target column
})
# Let's assume you want to predict the 'Target' based on other attributes
X = data.drop(['Target', 'Marital Status'], axis=1)  # Feature columns (excluding target and marital status)
y = data['Target']  # Target column (the 'Target' column with Positive/Negative)
# Print the first 5 rows of features and target
print("The first 5 values of data are:")
print(data.head())
# Print the first 5 values of the features
print("\nThe first 5 values of train data are:")
print(X.head())
# Print the first 5 values of the target
print("\nThe first 5 values of train output are:")
print(y.head())
# Convert categorical variables to numeric using LabelEncoder
le_age = LabelEncoder()
X['Age'] = le_age.fit_transform(X['Age'])
le_gender = LabelEncoder()
X['Gender'] = le_gender.fit_transform(X['Gender'])
le_education = LabelEncoder()
X['Education'] = le_education.fit_transform(X['Education'])
le_employment = LabelEncoder()
X['Employment'] = le_employment.fit_transform(X['Employment'])
le_location = LabelEncoder()
X['Location'] = le_location.fit_transform(X['Location'])
# Check the transformed features
print("\nNow the Train data is:")
print(X.head())
# Convert target variable 'Target' into numeric
```

```
le_target = LabelEncoder()

y = le_target.fit_transform(y)

# Check the transformed target

print("\nNow the Train output is:")

print(y)

# Split the data into training and testing sets

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42)

# Initialize and train the Naive Bayes classifier

classifier = GaussianNB()

classifier.fit(X_train, y_train)

# Evaluate the classifier's accuracy

y_pred = classifier.predict(X_test)

print("\nAccuracy is:", accuracy_score(y_test, y_pred))
```

OUTPUT :

```
The first 5 values of data are:
      Age  Gender Education Employment  Location Marital Status    Target
0   Young    Male  Bachelor  Full-time     Urban         Single  Positive
1  Middle  Female    Master  Part-time  Suburban        Married  Negative
2     Old    Male       PhD    Retired     Rural        Widowed  Negative
3   Young  Female  Bachelor  Full-time     Urban         Single  Positive

The first 5 values of train data are:
      Age  Gender Education Employment  Location
0   Young    Male  Bachelor  Full-time     Urban
1  Middle  Female    Master  Part-time  Suburban
2     Old    Male       PhD    Retired     Rural
3   Young  Female  Bachelor  Full-time     Urban

The first 5 values of train output are:
0    Positive
1    Negative
2    Negative
3    Positive
Name: Target, dtype: object

Now the Train data is:
   Age  Gender  Education  Employment  Location
0    2       1          0           0         2
1    0       0          1           1         1
2    1       1          2           2         0
3    2       0          0           0         2

Now the Train output is:
[1 0 0 1]

Accuracy is: 1.0
```

Nomenclature	Max. Marks	Marks Obtained
In Lab Performance (Pre-Viva, Execution, Result and Output)	05	5
Post Viva	05	5
Total	10	10
Faculty Signature		

RESULT :

Thus, the program to implement naïve Bayesian classifier Algorithm is executed successfully.

EX.NO : 7	NAÏVE BAYESIAN CLASSIFICATION MODEL
DATE : 14.03.25	

AIM :

To implement naïve Bayesian classification model using a dataset.

ALGORITHM :

Step 1 - Load and preprocess the dataset, converting categorical features into a combined text message.

Step 2 - Map the target labels ("Positive" and "Negative") to numeric values and split the dataset into training and test sets.

Step 3 - Transform the text data into a bag-of-words representation using CountVectorizer.

Step 4 - Train a Naive Bayes classifier (MultinomialNB) on the transformed training data.

Step 5 - Evaluate the model on the test set, calculating accuracy, precision, recall, and the confusion matrix.

PROGRAM :

```
import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.feature_extraction.text import CountVectorizer

from sklearn.naive_bayes import MultinomialNB

from sklearn.metrics import accuracy_score, confusion_matrix, precision_score,
recall_score

# Sample dataset

data = {

    'Age': ['Young', 'Middle', 'Old', 'Young'],

    'Gender': ['Male', 'Female', 'Male', 'Female'],

    'Education': ['Bachelor', 'Master', 'PhD', 'Bachelor'],

    'Employment': ['Full-time', 'Part-time', 'Retired', 'Full-time'],

    'Location': ['Urban', 'Suburban', 'Rural', 'Urban'],

    'Marital Status': ['Single', 'Married', 'Widowed', 'Single'],

    'Target': ['Positive', 'Negative', 'Negative', 'Positive']

}
```

```
# Creating DataFrame

df = pd.DataFrame(data)

# Let's create the text data by combining all features into one string (just for the sake of the example)

df['message'] = df['Age'] + ' ' + df['Gender'] + ' ' + df['Education'] + ' ' + df['Employment'] + ' ' + df['Location'] + ' ' + df['Marital Status']

# Output the total instances of the dataset

print("Total Instances of Dataset: ", df.shape[0])

# Map the target labels to numeric values

df['labelnum'] = df['Target'].map({'Positive': 1, 'Negative': 0})

# Define features (X) and target (y)

X = df['message']

y = df['labelnum']

# Split the data into training and test sets

Xtrain, Xtest, ytrain, ytest = train_test_split(X, y)

# Convert text data into a bag of words (CountVectorizer)

count_v = CountVectorizer()

Xtrain_dm = count_v.fit_transform(Xtrain)

Xtest_dm = count_v.transform(Xtest)

# Convert the transformed training data to a DataFrame for inspection

# Convert the transformed training data to a DataFrame for inspection

df_train=pd.DataFrame(Xtrain_dm.toarray(),
columns=count_v.get_feature_names_out())

# Print the first 5 rows of the transformed training data

print(df_train.head())# Print the first 5 rows of the transformed training data

print(df_train.head())

# Create and train the Naive Bayes classifier

clf = MultinomialNB()

clf.fit(Xtrain_dm, ytrain)

# Predict on the test data

pred = clf.predict(Xtest_dm)
```

```
# Print predictions for the test set
for doc, p in zip(Xtest, pred):
    p = 'Positive' if p == 1 else 'Negative'
    print(f"{doc} -> {p}")
# Evaluate the model performance
print('Accuracy Metrics: \n')
print('Accuracy: ', accuracy_score(ytest, pred))
print('Recall: ', recall_score(ytest, pred))
print('Precision: ', precision_score(ytest, pred))
print('Confusion Matrix: \n', confusion_matrix(ytest, pred))
```

OUTPUT :

```
Total Instances of Dataset:  4
   bachelor  female  full  male  old  phd  retired  rural  single  time  \
0         0       0     0     1    1    1        1      1       0     0
1         1       1     1     0    0    0        0      0       1     1
2         1       0     1     1    0    0        0      0       1     1

   urban  widowed  young
0      0        1      0
1      1        0      1
2      1        0      1
   bachelor  female  full  male  old  phd  retired  rural  single  time  \
0         0       0     0     1    1    1        1      1       0     0
1         1       1     1     0    0    0        0      0       1     1
2         1       0     1     1    0    0        0      0       1     1

   urban  widowed  young
0      0        1      0
1      1        0      1
2      1        0      1
Middle Female Master Part-time Suburban Married -> Positive
Accuracy Metrics:

Accuracy:  0.0
Recall:  0.0
Precision:  0.0
Confusion Matrix:
 [[0 1]
 [0 0]]
```

Nomenclature	Max. Marks	Marks Obtained
In Lab Performance (Pre-Viva, Execution, Result and Output)	05	05
Post Viva	05	05
Total	10	10
Faculty Signature		

RESULT :

Thus, the program to implement naïve Bayesian classification model is executed successfully.

EX.NO : 8	BAYESIAN CLASSIFICATION USING MEDICAL DATA
DATE : 21.03.25	

AIM :

To implement naïve Bayesian classification using a medical dataset.

ALGORITHM :

Step 1 - Install the required library pgmpy and import necessary libraries like numpy, pandas, and pgmpy.

Step 2 - Load the dataset (heartdisease.csv) and create a DataFrame.

Step 3 - Define and create the Bayesian Network model by specifying relationships between variables.

Step 4 - Fit the Bayesian Network model using MaximumLikelihoodEstimator and perform inference using VariableElimination.

Step 5 - Take user input for various features and predict the likelihood of heart disease using the trained model.

PROGRAM :

```
# Install required library

!pip install pgmpy

# Import necessary libraries

import numpy as np

import csv

import pandas as pd

from pgmpy.models import BayesianNetwork

from pgmpy.estimators import MaximumLikelihoodEstimator

from pgmpy.inference import VariableElimination

import pandas.util.testing as tm

# Load dataset

data = pd.read_csv("heartdisease.csv")

heart_disease = pd.DataFrame(data)

print(heart_disease)

# Create Bayesian Network model
```

```
from pgmpy.models import BayesianModel
model = BayesianModel([
    ('age', 'Lifestyle'),
    ('Gender', 'Lifestyle'),
    ('Family', 'heartdisease'),
    ('diet', 'cholestrol'),
    ('Lifestyle', 'diet'),
    ('cholestrol', 'heartdisease'),
    ('diet', 'cholestrol')
])
# Fit the model using MaximumLikelihoodEstimator
from pgmpy.estimators import MaximumLikelihoodEstimator
model.fit(heart_disease, estimator=MaximumLikelihoodEstimator)
# Perform inference using VariableElimination
from pgmpy.inference import VariableElimination
HeartDisease_infer = VariableElimination(model)
# Take user input for various features
print('For age Enter { SuperSeniorCitizen:0, SeniorCitizen:1, MiddleAged:2, Youth:3,
Teen:4 }')
print('For Gender Enter { Male:0, Female:1 }')
print('For Family History Enter { yes:1, No:0 }')
print('For diet Enter { High:0, Medium:1 }')
print('For lifeStyle Enter { Athlete:0, Active:1, Moderate:2, Sedentary:3 }')
print('For cholesterol Enter { High:0, BorderLine:1, Normal:2 }')
q = HeartDisease_infer.query(variables=['heartdisease'], evidence={
    'age': int(input('Enter age :')),
    'Gender': int(input('Enter Gender :')),
    'Family': int(input('Enter Family history :')),
    'diet': int(input('Enter diet :')),
    'Lifestyle': int(input('Enter Lifestyle :')),
    'cholestrol': int(input('Enter cholestrol :'))
```

```
})
# Print the result for heart disease prediction
print(q['heartdisease'])
```

OUTPUT :

	age	Gender	Family	diet	Lifestyle	cholestrol	heartdisease
0	0	0	1	1	3	0	1
1	0	1	1	1	3	0	1
2	1	0	0	0	2	1	1
3	4	0	1	1	3	2	0
4	3	1	1	0	0	2	0
5	2	0	1	1	1	0	1
6	4	0	1	0	2	0	1
7	0	0	1	1	3	0	1
8	3	1	1	0	0	2	0
9	1	1	0	0	0	2	1
10	4	1	0	1	2	0	1
11	4	0	1	1	3	2	0
12	2	1	0	0	0	0	0
13	2	0	1	1	1	0	1
14	3	1	1	0	0	1	0
15	0	0	1	0	0	2	1
16	1	1	0	1	2	1	1
17	3	1	1	1	0	1	0
18	4	0	1	1	3	2	0

```
For age Enter { SuperSeniorCitizen:0, SeniorCitizen:1, MiddleAged:2, Youth:3, Teen:4 }
For Gender Enter { Male:0, Female:1 }
For Family History Enter { yes:1, No:0 }
For diet Enter { High:0, Medium:1 }
For lifeStyle Enter { Athlete:0, Active:1, Moderate:2, Sedentary:3 }
For cholesterol Enter { High:0, BorderLine:1, Normal:2 }
Enter age :1
Enter Gender :1
Enter Family history :0
Enter diet :1
Enter Lifestyle :0
Enter cholestrol :1
+----------------+---------------------+
| heartdisease   |   phi(heartdisease) |
+================+=====================+
| heartdisease_0 |              0.0000 |
+----------------+---------------------+
| heartdisease_1 |              1.0000 |
+----------------+---------------------+
```

Nomenclature	Max. Marks	Marks Obtained
In Lab Performance (Pre-Viva, Execution, Result and Output)	05	05
Post Viva	05	05
Total	10	10
Faculty Signature		

RESULT :

Thus, the program to implement naïve Bayesian classification using medical dataset is executed successfully.

EXP NO: 9	ARTIFICIAL NEURAL NETWORK BY IMPLEMENTING THE BACKPROPAGATION ALGORITHM
DATE:04/04/2025	

AIM:

To build an Artificial Neural Network by implementing the Backpropagation algorithm

ALGORITHM :

STEP-1: Initialize weights randomly for input → hidden and hidden → output layers.

STEP-2: Forward pass: Compute outputs using sigmoid activations through layers.

STEP-3: Calculate error between predicted and actual outputs.

STEP-4: Backpropagate error: Compute gradients and adjust weights.

STEP-5: Repeat training loop to minimize loss and improve accuracy.

PROGRAM

```
import numpy as np
X = np.array(([2, 9], [1, 5], [3, 6]), dtype=float) # X = (hours sleeping, hours studying)
y = np.array(([92], [86], [89]), dtype=float) # y = score on test
# scale units
X = X/np.amax(X, axis=0) # maximum of X array
y = y/100 # max test score is 100
class Neural_Network(object):
def __init__(self):
# Parameters
self.inputSize = 2
self.outputSize = 1
self.hiddenSize = 3
# Weights
self.W1 = np.random.randn(self.inputSize, self.hiddenSize) # (3x2) weight matrix from
input to hidden layer
self.W2 = np.random.randn(self.hiddenSize, self.outputSize) # (3x1) weight matrix from
hidden to output layer
def forward(self, X):
```

```
#forward propagation through our network
self.z = np.dot(X, self.W1) # dot product of X (input) and first set of 3x2 weights
self.z2 = self.sigmoid(self.z) # activation function
self.z3 = np.dot(self.z2, self.W2) # dot product of hidden layer (z2) and second set of 3
x1 weights
o = self.sigmoid(self.z3) # final activation function
return o
def sigmoid(self, s):
return 1/(1+np.exp(-s)) # activation function
def sigmoidPrime(self, s):
return s * (1 - s) # derivative of sigmoid
def backward(self, X, y, o):
# backward propgate through the network
self.o_error = y - o # error in output
self.o_delta = self.o_error*self.sigmoidPrime(o) # applying derivative of sigmoid to
self.z2_error = self.o_delta.dot(self.W2.T) # z2 error: how much our hidden layer weights
contributed to output error
self.z2_delta = self.z2_error*self.sigmoidPrime(self.z2) # applying derivative of sigmoid to
z2 error

self.W1 += X.T.dot(self.z2_delta) # adjusting first set (input --> hidden) weights
self.W2 += self.z2.T.dot(self.o_delta) # adjusting second set (hidden --> output) weights
def train (self, X, y):
o = self.forward(X)
self.backward(X, y, o)
NN = Neural_Network()
for i in range(1000): # trains the NN 1,000 times
print ("\nInput: \n" + str(X))
print ("\nActual Output: \n" + str(y))
print ("\nPredicted Output: \n" + str(NN.forward(X)))
```

```
print ("\nLoss: \n" + str(np.mean(np.square(y - NN.forward(X))))) # mean sum squared loss
)
NN.train(X, y)
```

OUTPUT:

```
Streaming output truncated to the last 5000 lines.
[1. 0.66666667]]
Actual Output:
[[0.92]
[0.86]
[0.89]]
Predicted Output:
[[0.90256559]
[0.87418305]
[0.89241012]]
Actual Output:
[[0.92]
[0.86]
[0.89]]
Predicted Output:
[[0.90306431]
[0.87315843]
[0.89271695]]
Loss:
0.00015578115617263994
Input:
[[0.66666667 1. ]
[0.33333333 0.55555556]
[1. 0.66666667]]
```

Actual Output:

[[0.92]

[0.86]

[0.89]]

Predicted Output:

[[0.90306606]

[0.87315502]

[0.89271793]]

Loss:

0.00015573325001462293

```
        self.o_delta = self.o_error * self.sigmoidPrime(o)
        self.z2_error = self.o_delta.dot(self.w2.T)
        self.z2_delta = self.z2_error * self.sigmoidPrime(self.z2)

self.w1 += x.T.dot(self.z2_delta)
self.w2 += self.z2.T.dot(self.o_delta)
def train(self, x, y):
        o = self.forward(x)
        self.backward(x, y, o)
NN = Nearal_Network()
for i in range(1000):
        print("\n Input: \n" + str(x)) print("\n Actual output: "
+str(y))
        print("\n Predicted Output: \n" + str(NN.forward(x)))
        print("\n Loss: \n" + str(np.mean(np.square(y - NN.forward
(x)))))
```

Nomenclature	Max. Marks	Marks Obtained
In Lab Performance (Pre-Viva, Execution, Result and Output)	05	5
Post Viva	05	5
Total	10	10
Faculty Signature		

Result:

Thus the above program is executed & verified successfully

RESULT:

Thus, the program to build an Artificial Neural Network by implementing the Backpropagation algorithm .

EXP NO: 10	SVM TRAINING ALGORITHM
DATE:18/04/2025	

AIM:

To Build an SVM training algorithm that predicts whether each marked data belongs to one of two categories in a real-time dataset.

ALGORITHM :

Step 1: Import required libraries and load the dataset from CSV using pandas.

Step 2: Explore the dataset using .info() and .describe() to understand the structure and statistical properties.

Step 3: Visualize the data using pair plots, heatmaps, and scatter plots to analyze feature relationships and correlations.

Step 4: Split the dataset into features (X) and labels (y), and then into training and testing sets using train_test_split().

Step 5: Train a Support Vector Machine (SVM) model using the training data, predict on test data, and evaluate performance using accuracy, confusion matrix, and classification report.

PROGRAM

```
#import required libraries

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

%matplotlib inline

import seaborn as sns

#import models from scikit learn module:

from sklearn.model_selection import train_test_split

from sklearn import metrics

from sklearn.svm import SVC

#import Data

df_cancer = pd.read_csv('Breast_cancer_data.csv')

df_cancer.head()

#get some information about our Data-Set
```

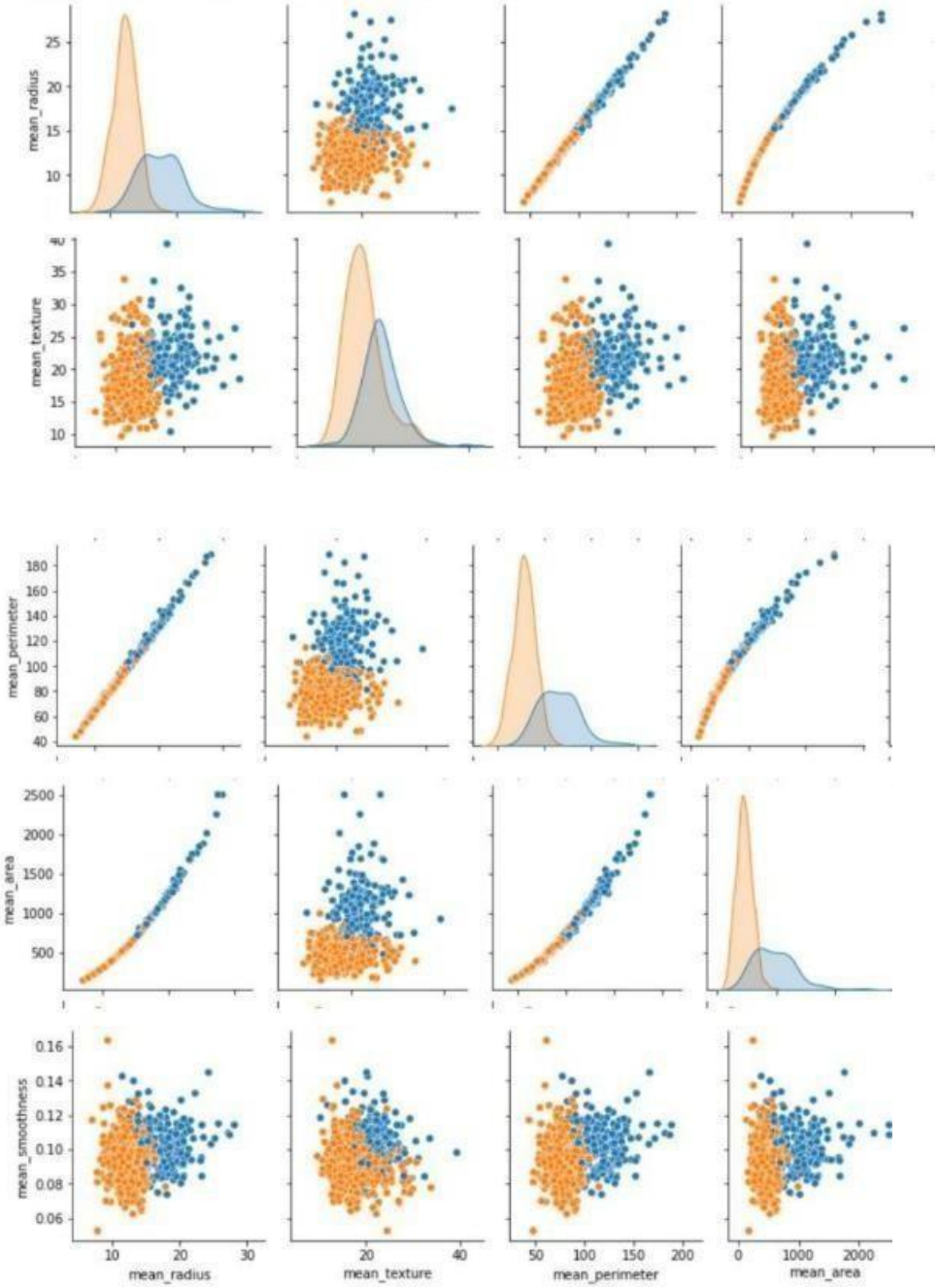
```
df_cancer.info()
df_cancer.describe()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 6 columns):
# Column Non-Null Count Dtype
---- -------- ----------------- ------
0 mean_radius 569 non-null float64
1 mean_texture 569 non-null float64
2 mean_perimeter 569 non-null float64
3 mean_area 569 non-null float64
4 mean_smoothness 569 non-null float64
5 diagnosis 569 non-null int64
dtypes: float64(5), int64(1)
memory usage: 26.8 KB
#visualizing data
sns.pairplot(df_cancer, hue = 'diagnosis')
plt.figure(figsize=(7,7))
sns.heatmap(df_cancer['mean_radius mean_texture mean_perimeter mean_area
mean_smoothness diagnosis'.split()].corr(), annot=True)
sns.scatterplot(x = 'mean_texture', y = 'mean_perimeter', hue = 'diagnosis', data =
df_cancer)
```

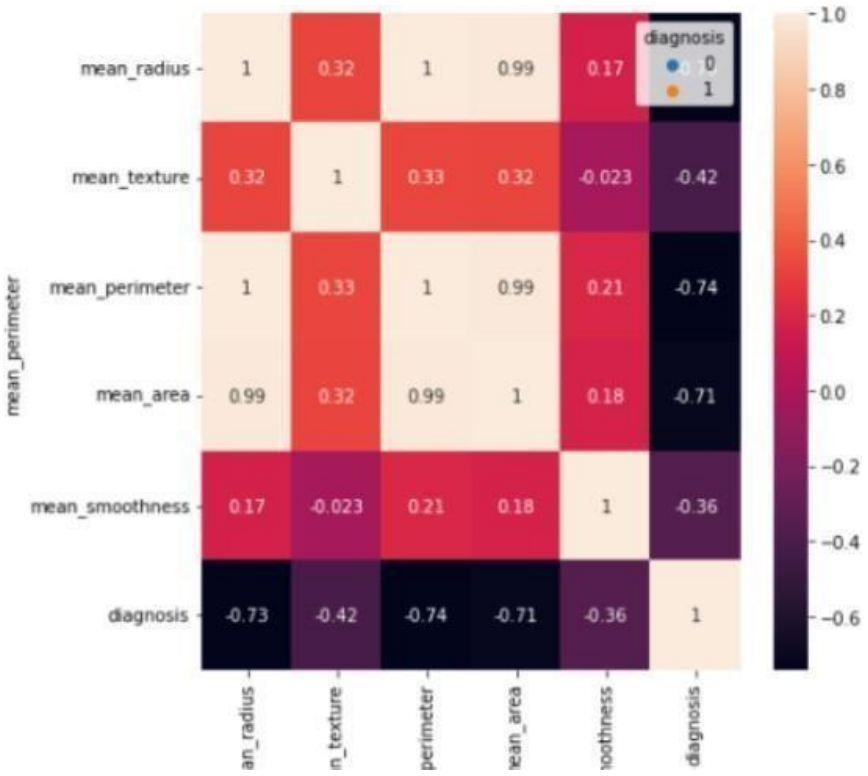
OUTPUT:

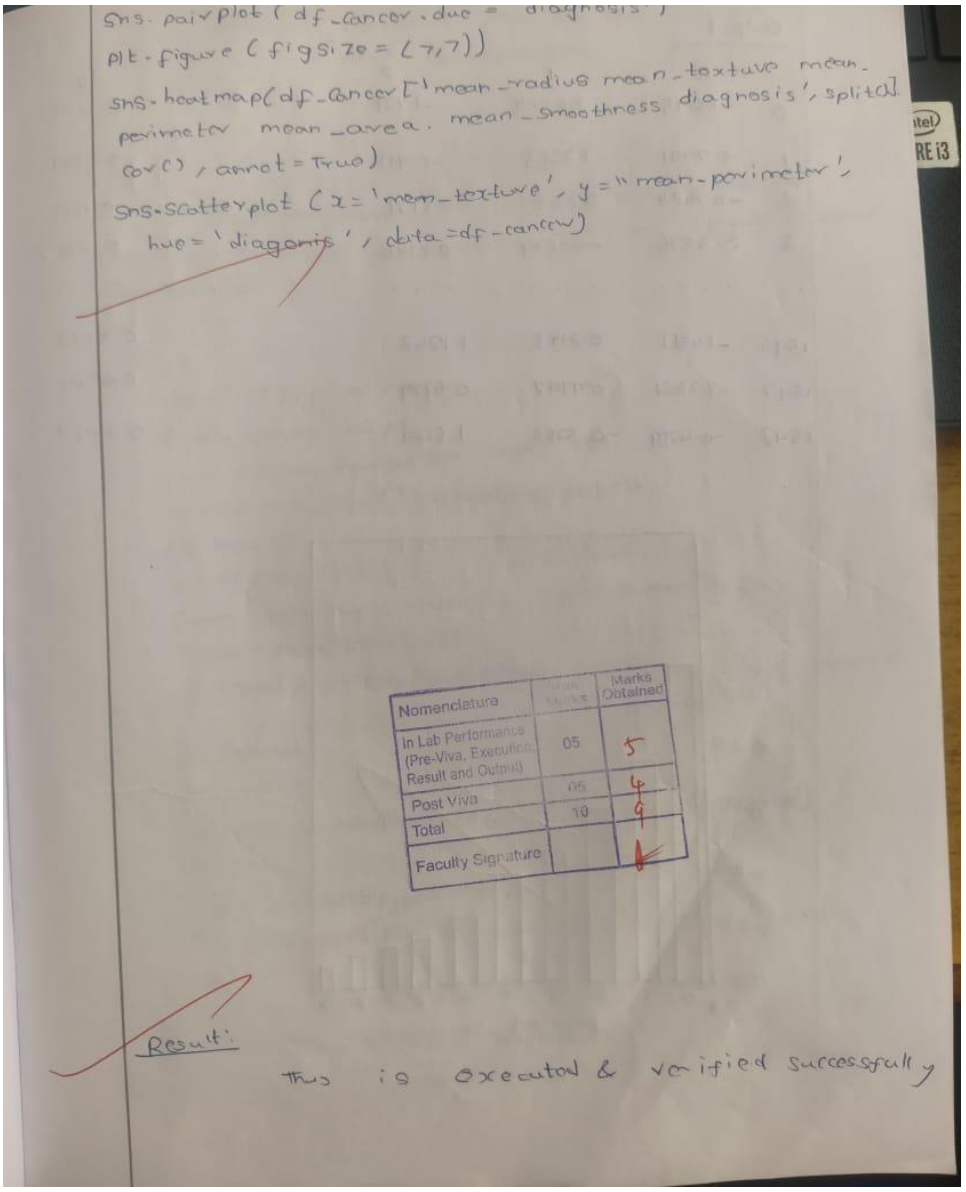
	mean_radius	mean_texture	mean_perimeter	mean_area	mean_smoothness	diagnosis
0	17.99	10.38	122.80	1001.0	0.11840	0
1	20.57	17.77	132.90	1326.0	0.08474	0
2	19.69	21.25	130.00	1203.0	0.10960	0
3	11.42	20.38	77.58	386.1	0.14250	0
4	20.29	14.34	135.10	1297.0	0.10030	0

	mean_radius	mean_texture	mean_perimeter	mean_area	mean_smoothness	diagnosis
count	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000
mean	14.127292	19.289649	91.969033	654.889104	0.096360	0.627417
std	3.524049	4.301036	24.298981	351.914129	0.014064	0.483918
min	6.981000	9.710000	43.790000	143.500000	0.052630	0.000000
25%	11.700000	16.170000	75.170000	420.300000	0.086370	0.000000
50%	13.370000	18.840000	86.240000	551.100000	0.095870	1.000000
75%	15.780000	21.800000	104.100000	782.700000	0.105300	1.000000
max	28.110000	39.280000	188.500000	2501.000000	0.163400	1.000000

```
<matplotlib.axes._subplots.AxesSubplot at 0x7f9f2e8a0b10>
```




diagnosis
0
1
mean_radius
mean_texture
mean_perimeter
mean_area
mean_smoothness
diagnosis
1 0.32 1 0.99 0.17
0.32 1 0.33 0.32 -0.023 -0.42
1 0.33 1 0.99 0.21 -0.74
0.99 0.32 0.99 1 0.18 -0.71
0.17 -0.023 0.21 0.18 1 -0.36
-0.73 -0.42 -0.74 -0.71 -0.36 1
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4
−0.6



```
sns.pairplot(df_cancer, hue = 'diagnosis')
plt.figure(figsize=(7,7))
sns.heatmap(df_cancer['mean_radius mean_texture mean_perimeter mean_area. mean_smoothness diagnosis', split(0)].corr(), annot=True)
sns.scatterplot(x='mean_texture', y="mean_perimeter', hue='diagonis', data=df_cancer)
```

Nomenclature	Max Marks	Marks Obtained
In Lab Performance (Pre-Viva, Execution, Result and Output)	05	5
Post Viva	05	4
Total	10	9
Faculty Signature		

Result:

This is executed & verified successfully

RESULT:

Thus, the program to build an SVM training algorithm that predicts whether each marked data belongs to one of two categories in a real-time dataset was successful.

EXP NO: 11	PRINCIPAL COMPONENT ANALYSIS AND SINGULAR VALUE DECOMPOSITION.
DATE:25/04/2025	

AIM:

To demonstrate the working of the dimensionality reduction of the data using Principal Component Analysis and Singular Value Decomposition.

ALGORITHM :

Step 1: Import required libraries like numpy, pandas, matplotlib, sklearn, and scipy.

Step 2: Load the dataset (e.g., winequality-red.csv) using pandas, and separate the target column (quality) from the features.

Step 3: Normalize the feature data using StandardScaler() to standardize all columns.

Step 4: Apply Principal Component Analysis (PCA) on the scaled data and transform it into principal components.

Step 5: Visualize the explained variance ratio using a bar plot to identify which components capture the most variance.

Step 6: Define a sample high-dimensional matrix and perform Singular Value Decomposition (SVD) using svd() to get U, Σ, and V^T.

Step 7: Select top k singular values to construct a reduced Sigma matrix and reconstruct the approximate original matrix.

Step 8: Project the original matrix to a lower dimension using either T = U × Σ or T = A × V^T for further use in modeling or analysis.

PROGRAM

```
import numpy as np

import pandas as pd

from sklearn.decomposition import PCA

df = pd.read_csv('winequality-red.csv')

df.head()

y = df.pop('quality')

 from sklearn.preprocessing import StandardScaler

scalar = StandardScaler()

df_scaled = pd.DataFrame(scalar.fit_transform(df), columns=df.columns)

df_scaled
```

```
pca = PCA()
df_pca = pd.DataFrame(pca.fit_transform(df_scaled))
df_pca
import matplotlib.pyplot as plt
pd.DataFrame(pca.explained_variance_ratio_).plot.bar()
plt.legend('')
plt.xlabel('Principal Components')
plt.ylabel('Explained Varience');
```

SINGULAR VALUE DECOMPOSITION:

```
from numpy import array
from numpy import diag
from numpy import zeros
from scipy.linalg import svd
# define a matrix
A = array([
[1,2,3,4,5,6,7,8,9,10],
[11,12,13,14,15,16,17,18,19,20],
[21,22,23,24,25,26,27,28,29,30]])
print(A)
# Singular-value decomposition
U, s, VT = svd(A)
# create m x n Sigma matrix
Sigma = zeros((A.shape[0], A.shape[1]))
# populate Sigma with n x n diagonal matrix
Sigma[:A.shape[0], :A.shape[0]] = diag(s)
# select
n_elements = 2
Sigma = Sigma[:, :n_elements]
```

```
VT = VT[:n_elements, :]
# reconstruct
B = U.dot(Sigma.dot(VT))
print(B)
# transform
T = U.dot(Sigma)
print(T)
T = A.dot(VT.T)
print(T)
```

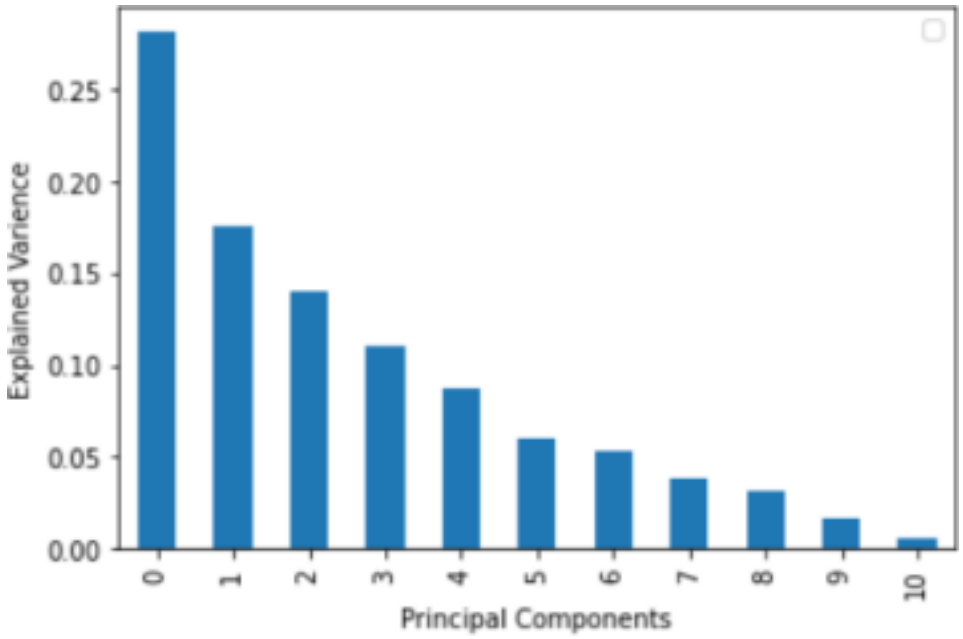
OUTPUT:

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol
0	-0.528360	0.961877	-1.391472	-0.453216	-0.243707	-0.466193	-0.379133	0.558274	1.288643	-0.579207	-0.960246
1	-0.298547	1.967442	-1.391472	0.043416	0.223875	0.872638	0.624363	0.028261	-0.719933	0.128950	-0.584777
2	-0.298547	1.297965	-1.186070	-0.169427	0.096353	-0.083668	0.229047	0.134264	-0.331177	0.048369	-0.584777
3	1.654856	-1.384643	1.484154	-0.453216	-0.264960	0.107592	0.411900	0.664277	-0.979104	-0.461180	-0.584777
4	-0.528360	0.961877	-1.391472	-0.453216	-0.243707	-0.466193	-0.379133	0.558274	1.288643	-0.579207	-0.960246
...	...	...	...	...	...	...	...	...	...	...	...
1594	-1.217796	0.403229	-0.980669	-0.362271	0.053643	1.542054	-0.075643	-0.370759	0.899889	-0.461180	0.072294
1595	-1.390155	0.123905	-0.877568	-0.240375	-0.541259	2.211469	0.137820	-0.862192	1.353436	0.601036	0.729364
1596	-1.160543	-0.099654	-0.723916	-0.169427	-0.243707	1.255161	-0.156879	-0.553554	0.709568	0.542042	0.541630
1597	-1.390155	0.654620	-0.775267	-0.362271	-0.264960	1.542054	-0.075643	-0.676457	1.677400	0.305990	-0.209030
1598	-1.332702	-1.216849	1.031999	0.702894	-0.434990	0.203223	-0.108861	-0.666097	0.811139	0.015924	0.641630

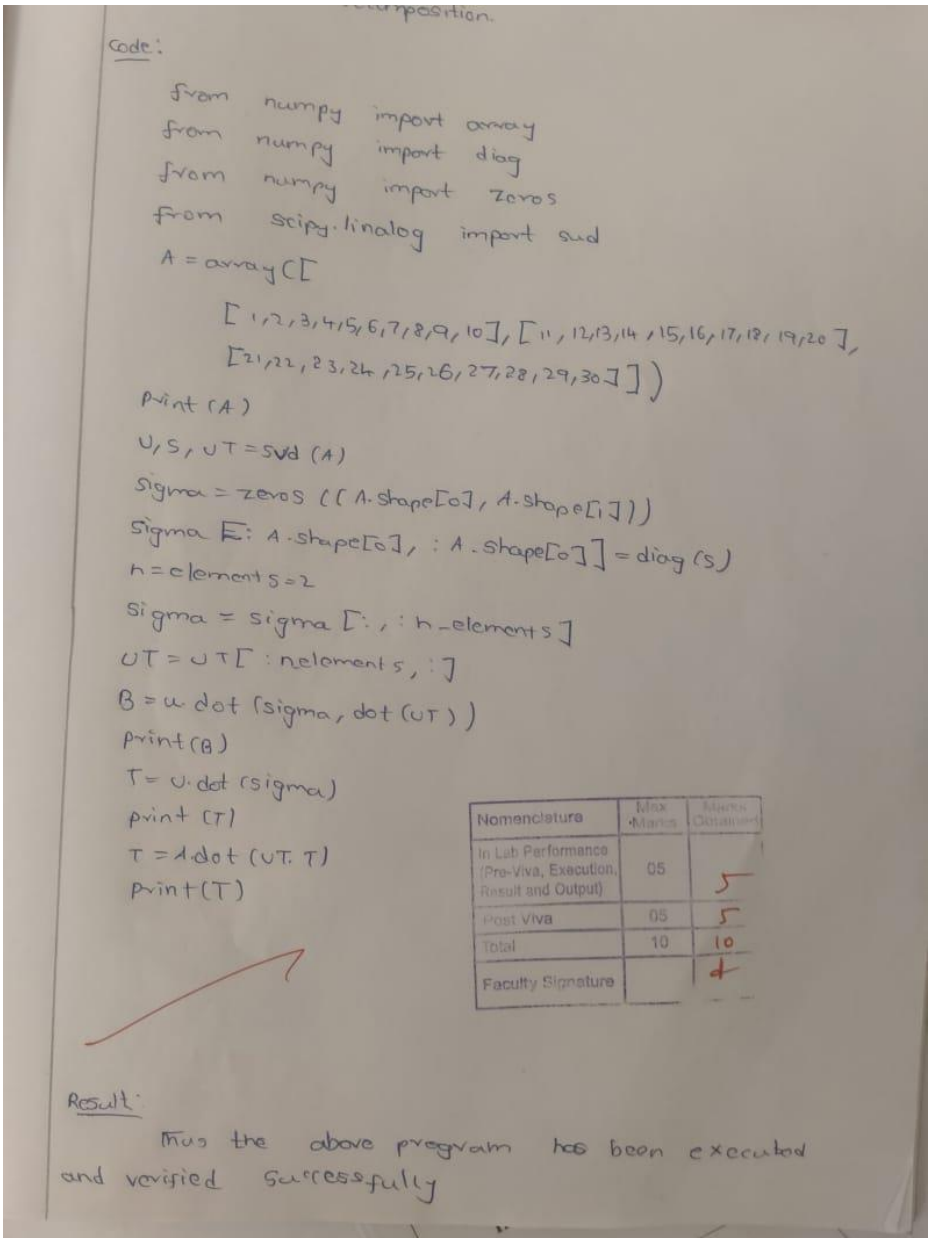
1599 rows × 11 columns

	0	1	2	3	4	5	6	7	8	9	10
0	-1.619530	0.450950	-1.774454	0.043740	0.067014	-0.913921	-0.161043	-0.282258	0.005098	-0.267759	0.048630
1	-0.799170	1.856553	-0.911690	0.548066	-0.018392	0.929714	-1.009829	0.762587	-0.520707	0.062833	-0.038142
2	-0.748479	0.882039	-1.171394	0.411021	-0.043531	0.401473	-0.539553	0.597946	-0.086857	-0.187442	-0.118229
3	2.357673	-0.269976	0.243489	-0.928450	-1.499149	-0.131017	0.344290	-0.455375	0.091577	-0.130393	0.316714
4	-1.619530	0.450950	-1.774454	0.043740	0.067014	-0.913921	-0.161043	-0.282258	0.005098	-0.267759	0.048630
...	...	...	...	...	...	...	...	...	...	...	...
1594	-2.150500	0.814286	0.617063	0.407687	-0.240936	0.054835	0.170812	-0.355866	-0.971524	0.356851	-0.053382
1595	-2.214496	0.893101	1.807402	0.414003	0.119592	-0.674711	-0.607970	-0.247640	-1.058135	0.478879	-0.241258
1596	-1.456129	0.311746	1.124239	0.491877	0.193716	-0.506410	-0.231082	0.079382	-0.808773	0.242248	-0.302910
1597	-2.270518	0.979791	0.627965	0.639770	0.067735	-0.860408	-0.321487	-0.468876	-0.612248	0.779404	0.040923
1598	-0.426975	-0.536690	1.628955	-0.391716	0.450482	-0.496154	1.189132	0.042176	0.404309	0.779440	-0.449781

1599 rows × 11 columns




```
[[ 1  2  3  4  5  6  7  8  9 10]
 [11 12 13 14 15 16 17 18 19 20]
 [21 22 23 24 25 26 27 28 29 30]]
[[ 1.  2.  3.  4.  5.  6.  7.  8.  9. 10.]
 [11. 12. 13. 14. 15. 16. 17. 18. 19. 20.]
 [21. 22. 23. 24. 25. 26. 27. 28. 29. 30.]]
[[-18.52157747   6.47697214]
 [-49.81310011   1.91182038]
 [-81.10462276  -2.65333138]]
[[-18.52157747   6.47697214]
 [-49.81310011   1.91182038]
 [-81.10462276  -2.65333138]]
```



Code:

```
from numpy import array
from numpy import diag
from numpy import zeros
from scipy.linalog import svd
A = array([
    [1,2,3,4,5,6,7,8,9,10], [11,12,13,14,15,16,17,18,19,20],
    [21,22,23,24,25,26,27,28,29,30]])
print(A)
U,S,VT=svd(A)
sigma = zeros((A.shape[0], A.shape[1]))
sigma [: A.shape[0], : A.shape[0]] = diag(S)
n=elements=2
sigma = sigma [:, : n_elements]
VT = VT[ : nelements, :]
B = u.dot(sigma, dot(VT))
print(B)
T= U.dot(sigma)
print (T)
T = A.dot(VT. T)
print(T)
```

Nomenclature	Max Marks	Marks Obtained
In Lab Performance (Pre-Viva, Execution, Result and Output)	05	5
Post Viva	05	5
Total	10	10
Faculty Signature		

Result:

Thus the above program has been executed and verified successfully

RESULT:

Thus, to demonstrate the working of the dimensionality reduction of the data using Principal Component Analysis and Singular Value Decomposition was successful.

EX.NO :	
DATE :	**STUDY ON ANN, RNN, CNN**

AIM:

To study on Artificial Neural Networks [ANN], Recurrent Neural Networks [RNN] , Convolutional Neural Networks [CNN].

ARTIFICIAL NEURAL NETWORKS [ANN] :

ANN, is a group of multiple perceptrons or neurons at each layer. ANN is also known as a Feed-Forward Neural network because inputs are processed only in the forward direction. This type of neural networks are one of the simplest variants of neural networks. They pass information in one direction, through various input nodes, until it makes it to the output node. The network may or may not have hidden node layers, making their functioning more interpretable.

A single perceptron (or neuron) can be imagined as a Logistic Regression. Artificial Neural Network, or ANN, is a group of multiple perceptrons/ neurons at each layer. ANN is also known as a Feed-Forward Neural network because inputs are processed only in the forward direction.

ANN consists of 3 layers – Input, Hidden and Output. The input layer accepts the inputs, the hidden layer processes the inputs, and the output layer produces the result. Essentially, each layer tries to learn certain weights.

ANN can be used to solve problems related to:

- Tabular data
- Image data
- Text data

Artificial Neural Network is capable of learning any nonlinear function. Hence, these networks are popularly known as Universal Function Approximators. ANNs have the capacity to learn weights that map any input to the output.

One of the main reasons behind universal approximation is the activation function. Activation functions introduce nonlinear properties to the network. This helps the network learn any complex relationship between input and output.

Input Layer:

As the name suggests, it accepts inputs in several different formats provided by the programmer.

Hidden Layer:

The hidden layer presents in-between input and output layers. It performs all the calculations to find hidden features and patterns.

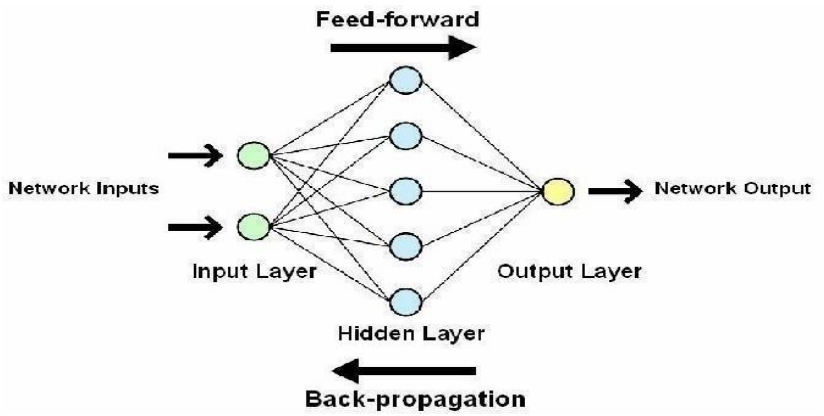
Output Layer:

The input goes through a series of transformations using the hidden layer, which finally results in output that is conveyed using this layer.

The artificial neural network takes input and computes the weighted sum of the inputs and includes a bias. This computation is represented in the form of a transfer function.

$$\sum_{i=1}^{n} Wi * Xi + b$$

It determines weighted total is passed as an input to an activation function to produce the output. Activation functions choose whether a node should fire or not. Only those who are fired make it to the output layer. There are distinctive activation functions available that can be applied upon the sort of task we are performing.



RECURRENT NEURAL NETWORKS [RNN] :

RNN is more complex. They save the output of processing nodes and feed the result back into the model (they did not pass the information in one direction only). This is how the model is said to learn to predict the outcome of a layer. Each node in the RNN model acts as a memory cell, continuing the computation and implementation of operations. If the network's prediction is incorrect, then the system self-learns and continues working towards the correct prediction during backpropagation.
RNN has a recurrent connection on the hidden state. This looping constraint ensures that sequential information is captured in the input data.

We can use recurrent neural networks to solve the problems related to:

- Time Series data
- Text data
- Audio data

RNN captures the sequential information present in the input data i.e. dependency between the words in the text while making predictions.

RNNs share the parameters across different time steps. This is popularly known as Parameter Sharing. This results in fewer parameters to train and decreases the computational cost.

Deep RNNs (RNNs with a large number of time steps) also suffer from the vanishing and exploding gradient problem which is a common problem in all the different types of neural networks.

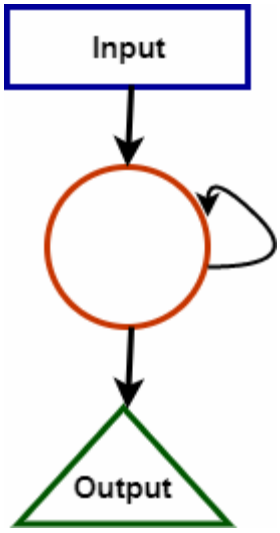
An RNN remembers each and every information through time. It is useful in time series prediction only because of the feature to remember previous inputs as well. This is called Long Short Term Memory.

Recurrent neural network are even used with convolutional layers to extend the effective pixel neighborhood.

It has Gradient vanishing and exploding problems.Training an RNN is a very difficult task.It cannot process very long sequences if using tanh or relu as an activation function.

The recurrent network first performs the conversion of independent activations into dependent ones. It also assigns the same weight and bias to all the layers, which reduces the complexity of RNN of parameters. And it provides a standard platform for memorization of the previous outputs by providing previous output as an input to the next layer.

These three layers having the same weights and bias, combine into a single recurrent unit.



For calculating the current state-

ht =f(ht-1, Xt)

Where ht= current state
Ht-1= previous state
Xt= input state

To apply the activation function tanh, we have-

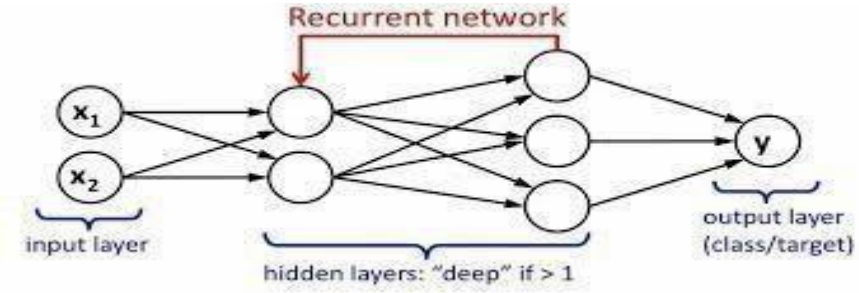
ht = tanh (Whhht-1+ WxhXt)

Where:

Whh = weight of recurrent neuron and,
Wxh = weight of the input neuron

The formula for calculating output:

Yt = Whyht



CONVOLUTIONAL NEURAL NETWORKS [CNN]:

CNN are one of the most popular models used today. This neural network computational model uses a variation of multilayer perceptrons and contains one or more convolutional layers that can be either entirely connected or pooled. These convolutional layers create feature maps that record a region of image which is ultimately broken into rectangles and sent out for nonlinear processing.

Convolutional neural networks (CNN) are all the rage in the deep learning community right now. These CNN models are being used across different applications and domains, and they're especially prevalent in image and video processing projects.

The building blocks of CNNs are filters or kernels. Kernels are used to extract the relevant features from the input using the convolution operation.

Though convolutional neural networks were introduced to solve problems related to image data, they perform impressively on sequential inputs as well.

CNN learns the filters automatically without mentioning it explicitly. These filters help in extracting the right and relevant features from the input data

CNN captures the spatial features from an image. Spatial features refer to the arrangement of pixels and the relationship between them in an image. They help us in identifying the object accurately, the location of an object, as well as its relation with other objects in an image.

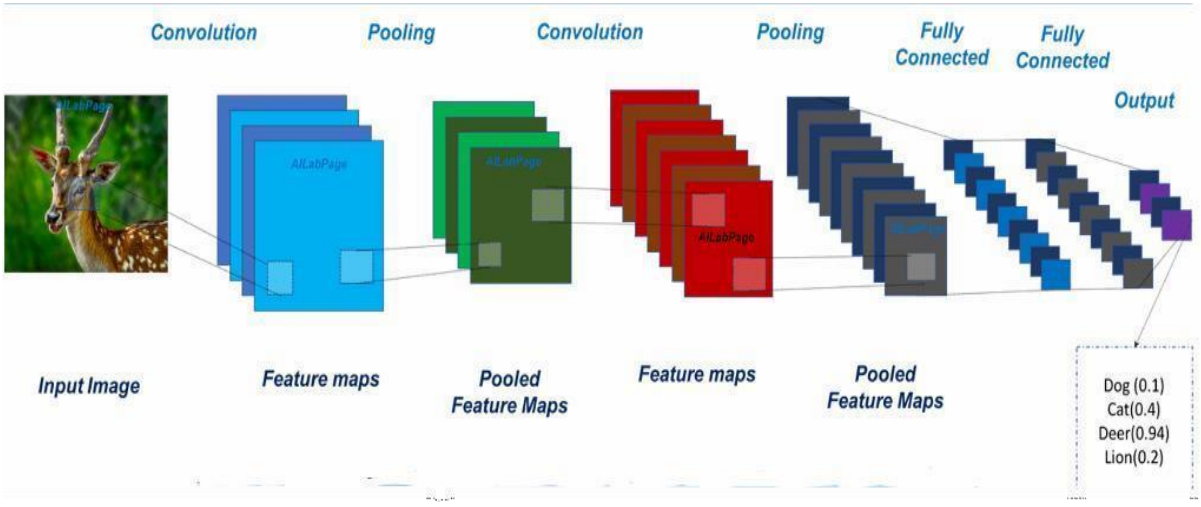
CNN also follows the concept of parameter sharing. A single filter is applied across different parts of an input to produce a feature map.

CNN has very high accuracy in image recognition problems. Automatically detects the important features without any human supervision and is weight sharing.

CNN do not encode the position and orientation of object. Its lack of ability to be spatially invariant to the input data and lots of training data is required.

In simple terms, a CNN is a feed forward neural network that applies a filter over the input signal to get a modified output signal. Feed forward neural networks are the simplest type of networks where the information moves ina single direction only; from the input nodes, to the hidden nodes (if any) and finally to the output layer. There are no loops or cycles in such a network.

CNNs are suitable for spatial data, such as images and is widely used in image and video processing. They can capture the spatial features, i.e. the arrangement and the relationships between the pixels in an image. This makes CNNs particularly useful in identifying objects, as well as their position within and in relation to other objects in an image.



RESULT:

Thus,the study on Artificial Neural Networks [ANN] , Recurrent Neural Networks [RNN] , Convolutional Neural Networks [CNN] were executed successfully.